Fuzzy Laboratory 1

Fuzzy Logic Enhanced Time Petri Nets (FLETPN) Model

1. Laboratory Objectives

- Acquiring the concepts
  - Fuzzy logic,
  - Fuzzyfication and defuzzyfication values,
  - Determining the fuzzy logic rules set FLRS,
  - FLETPN model,
- Developments and tests

2. Theoretical considerations

As is well known, the human being works according to some rules that is formulated through education and instinct, the use of concepts and measures is not always accurate but rather "more flexible" in quantification. These aspects led to fuzzy logic, theorized by Lotfi A. Zadeh in 1965. Fuzzy logic uses special variables, "fuzzy" values, thus forming a workspace that is different from the one characterized by the exact numerical values.

Fuzzy variables represent a class or set of x_i values, which are contained in a certain proportion in a set, the proportion is given by the function $\mu_i (x_i)$. The goal is to reduce the complexity of the outside world to a finite number of variables that can be intuitively operated. The way to work with these concepts or variables is in the form of rules of logical implication:

IF (*input* ***IS*** *{fuzzy_set1}*) **THEN** (*action* ***IS*** *{fuzzy_set2}*),

where "fuzzy_set1" is the set of values that "input" can take, and "fuzzy_set2", the set of possible values for "action".

Example:

IF (x_1 is A_1 ^ x_2 is A_2 ^ … ^ x_m is A_m) **THEN** (y_1 is A_1 ^ y_2 is A_2 ^ … ^ y_n is A_n),

where $x_i \in$ *fuzzy_set1* and $y_i \in$ *fuzzy_set2*.

Fuzzy values in “fuzzy_set” sets can be "LOW", "MEDIUM", "EXTREME", "HOT", "WARM", "CLOSE", "FAR" etc.

Next, we will use the fuzzy set FS = {NL, NM, ZR, PM, PL} with Negative Long, Medium Negative, Zero, Positive Medium and Positive Long for fuzzy_set1 and fuzzy_set2.

The operation that converts a real-world variable into a fuzzy variable is called **fuzzyfication** and is performed according to a membership function, see Fig. 2.1.

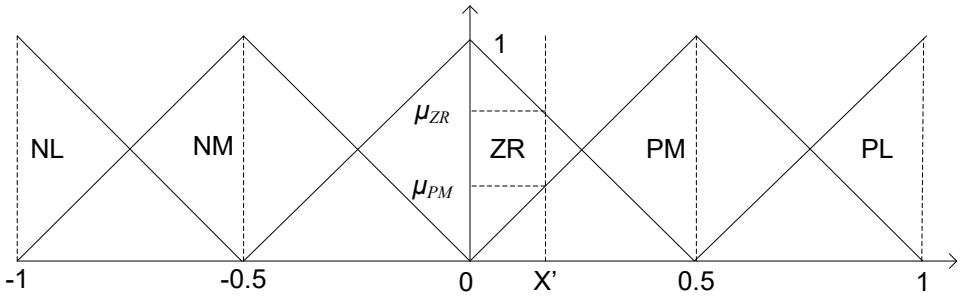


Figure 2.1 Fuzzyfication function of variables

Example:

For a variable x', the ZR membership function: $\mu_{ZR}(x') = 0.65$ and for the membership of PM: $\mu_{PM}(x') = 0.35$. Observe that: $\mu_{ZR}(x') + \mu_{PM}(x') = 1$.

The inverse operation that converst a fuzzy variable to a numeric value is called **defuzzyfication** and is performed similarly to fuzzyfication by calculating the position of the variable on the abscissa axis, depending on the degrees of the membership of each of the fuzzy values indicated. There exist several possible methods, the center of gravity method is usually used:

$$x = \frac{\sum_i x_i \cdot \mu_i(x_i)}{\sum_i \mu_i(x_i)}$$

where x_i are the values of the fuzzy levels in the universe of the variable x, $\mu_i(x_i)$ is the degree of the fuzzy variable membership at each of the fuzzy levels, and x is the crisp result of the defuzzy operation.

A rule can be met in a higher or lower proportion as its condition is met. Thus, several rules can be fulfilled at a given time, and so the value of the command values ("action") takes several fuzzy values in varying proportions.

Figure 2.2 presents the architecture of a control system that uses fuzzy logic. Inputs values that are measured by the sensors are fuzzy, the fuzzy logic control algorithm applied, then the output values are defuzzyfied and applied to the system, then the errors are measured and the cycle resumes.

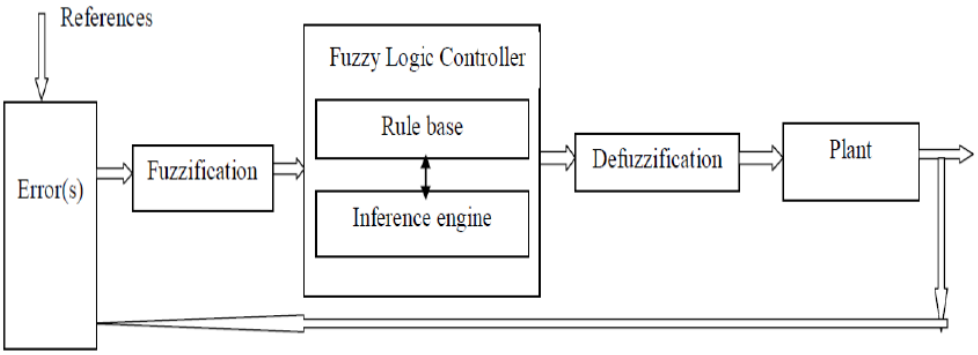


Figure 2.2 The architecture of a fuzzy logic control system

3. Fuzzy Logic Enhanced Time Petri Nets

Figure 3.1 is a simple component with the Fuzzy Logic Enhanced Time Petri Nets (FLETPN) with two inputs and two outputs. The domains $x_1, x_2, x_3, x_4 \in [-1,1]$, and we consider the amplification factors $w_1, w_2 \in [-10, 10]$. The amplification is used so that the resulting values x'_1 și x'_2 are within an acceptable range.

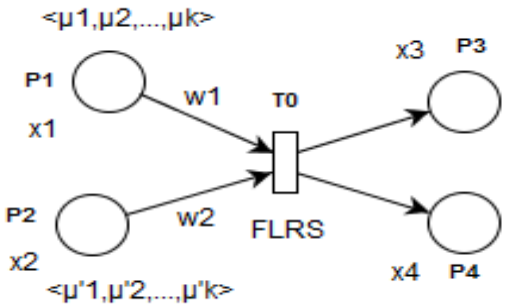


Figure 3.1 FLETPN example

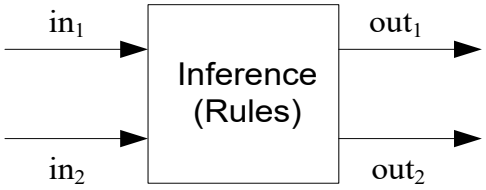


Figure 3.2 The FLETPN model

The model is capable to describe the synchronous and asynchronous reaction of a system to continuous and discreet changes. A place may contain a token that represents a synchronous or asynchronous reaction respectively to a continuous or a discreet change.

Each place has a variable x_i assigned. A token set in a place p_i represents the degrees of a variable's membership $\mu_i(x_i)$ to the fuzzy set FS. For a token in our example we have $n=5: <\mu_{NL}, \mu_{NM}, \mu_{ZR}\ \mu_{PM}, \mu_{PL}>$. Each arc has a w_i coefficient.

A transition is executable if and only if there is at least one rule in the corresponding table that can be enabled. If multiple rules can be enabled at any given time, all rules are used, the resulting tokens being the sum of the tokens. A normalization procedure is used so that the output location contains a token that meets the condition $\mu_{NL} + \mu_{NM} + \mu_{ZR} + \mu_{PM} + \mu_{PL} = 1$.

Execution of an executable transition t_i implies:

- extracting tokens from the input places, marked ${}^{o}t_j$;
- defuzzyfication of all the input variables x_i;
- multiplying the variables with the corresponding coefficient w_i;
- fuzzyfication of the varibles x_{ij};
- using FLRS with the input variables x_{ij};
- normalization of the operation that reduces the consequences to one and leads to the injection of a single token in the output places;
- Injection of the resulting tokens into the output places, marked $t^{o}_{j;}$

An example of the solutions that are calculated for two inputs and two outputs is given in Table 1:

Tabel 1: FLRS Example.

x_1/x_2	NL	NM	ZR	PM	PL
NL	PL,PL	PM,NL	ZR,ZR	NL,PL	ZR,PL
NM	PL,PM	NM,ZR	PL,NM	PL,NM	NM,PL
ZR	NL,PM	PL,NM	ZR,ZR	ZR,NM	PL,ZR
PM	ZR,PL	ZR,PM	NM,PM	PM,PM	NL,NL
PL	PM,ZR	PM,NM	ZR,ZR	NM,ZR	PL,NM

4. Fuzzyfication and defuzzification the values

Example:

Two input values are given x_1=0.3 and x_2=0.7 ϵ [-1, 1] and the coefficients w_1=2 şi w_2=3.

Required: Calculate outputs x_3 and x_4.

We multiply each variable by the appropriate coefficient:

$x'_1=x_1*w_1=0.3*2=0.6$ and

$x'_2=x_2*w_2=0.7*3=2.1=>x'_2=1$

Fuzzyifying the two values with the function in Figure 2.1, we have for x'_1:

$\mu_{PL}=(0.6-0.5)*1/0.5=0.2$ and $\mu_{PM}=1*(0.5-0.1)/0.5=0.8$ so the place p_1 will contain the values:

$\mu_{x'1}=<0, 0, 0, 0.8, 0.2>$

And for x'_2 and the place p_2:

$\mu_{x'2}$=<0, 0, 0, 0, 1>.

Considering the FLRS table in the above example, the rules are enabled:

r_1: <x_3,x_4> = <NL,NL> and r_2:<x_3,x_4> = <PL,NM > .

For the defuzzyfication we have:

The power of the rule and, if we have a conjunction, it is

$s_i = \mu_1 \cdot \mu_2$ or $s_i = min\{\mu_1, \mu_2\}$.

In our case s_1=0.8*1=0.8 and s_2=0.2*1=0.2.

x_3=(NL*0.8+PL*0.2)/(0.8+0.2)=((-1)*0.8+(1)*0.2)/1=(-0.8+0.2)/1= -0.6.

x_4=(NL*0.8+NM*0.2)/(0.8+0.2)=((-1)*0.8+(-0.5)*0.2)/1=(-0.8-0.1)/1= -0.9.

5. Develop and Test

5.1 Application 1

Specifications: Build an application is required to be constructed and build an inverter for the input values x_1, x_2, w_1, w_2 and calculate the output values x_3 and x_4.

In the mathematical model for the inverter, we consider:

$(x_1, x_2) \rightarrow (x_3, x_4)$ $x_3 = - x_1$; $x_4 = - x_2$; $w_1 = w_2 = 1$:

$x_3 = - w_1 \cdot x_1 \leftrightarrow w_1 \cdot x_1 \in [-1,1]$

$x_3 = 1; \leftrightarrow w_1 \cdot x_1 < -1;$

$x_3 = -1; \leftrightarrow w_1 \cdot x_1 > 1;$

$x_4 = - w_2 \cdot x_2 \leftrightarrow w_2 \cdot x_2 \in [-1,1]$

$x_4 = 1; \leftrightarrow w_2 \cdot x_2 < -1;$

$x_4 = -1; \leftrightarrow w_2 \cdot x_2 > 1;$

and the FLRS Table:

Table 2: Inversor FLRS Table

$x_1 \setminus x_2$	NL	NM	ZR	PM	PL
NL	PL, PL	PL, PM	PL, ZR	PL, NM	PL, NL
NM	PM, PL	PM, PM	PM, ZR	PM, NM	PM, NL
ZR	ZR, PL	ZR, PM	ZR, ZR	ZR, NM	ZR, NL
PM	NM, PL	NM, PM	NM, ZR	NM, NM	NM, NL
PL	NL, PL	NL, PM	NL, ZR	NL, NM	NL, NL

The class diagram

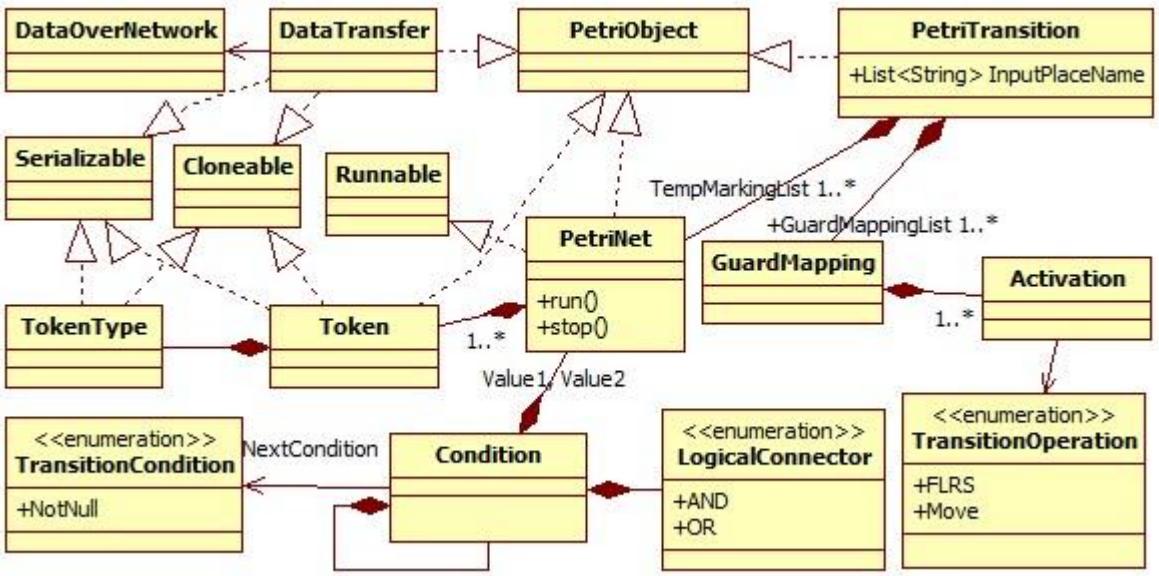


Figure 5.1 The All Petri Framework – FLETPN General class diagram

The FLETPN is implemented within the All Petri framework, figure 5.1 shows a general class diagram for the OETPN framework and figure 5.2 shows the FLETPN implementation where the TokenType is the Fuzzy class and the Token is DataFuzzy. The Fuzzy class has a Value of type float (the unfuzzified token) and the FuzzyVector which is the fuzzified token. The FLRS (Fuzzy Logic Rule Set)

consists of FLRSPart which has FV (FuzzyValue) and FZ (FuzzyValue). Each cell in the FLRS has it's index starts from the input token value (NL, NM,..,PL), the cell has a FV to represent the output token FZ.

Each FLETPN is an object that contains a list of places and a list of transitions. The transition must have one GuardMapping object containing the guard which is an instance of the Condition class that is connected to the next condition with the LogicalConnector (AND, OR), the Condition class compares between two values (tokens). The mapping is activated when the transition's condition is true, it is an instance of the Activation class that contains the TransitionOperations that are performed on the tokens to calculated the output places of the transition based on the FLRS or just simple Move operations. The marking is a list of places including the input and output channels, only the output channels are of type DataTrasnfer.

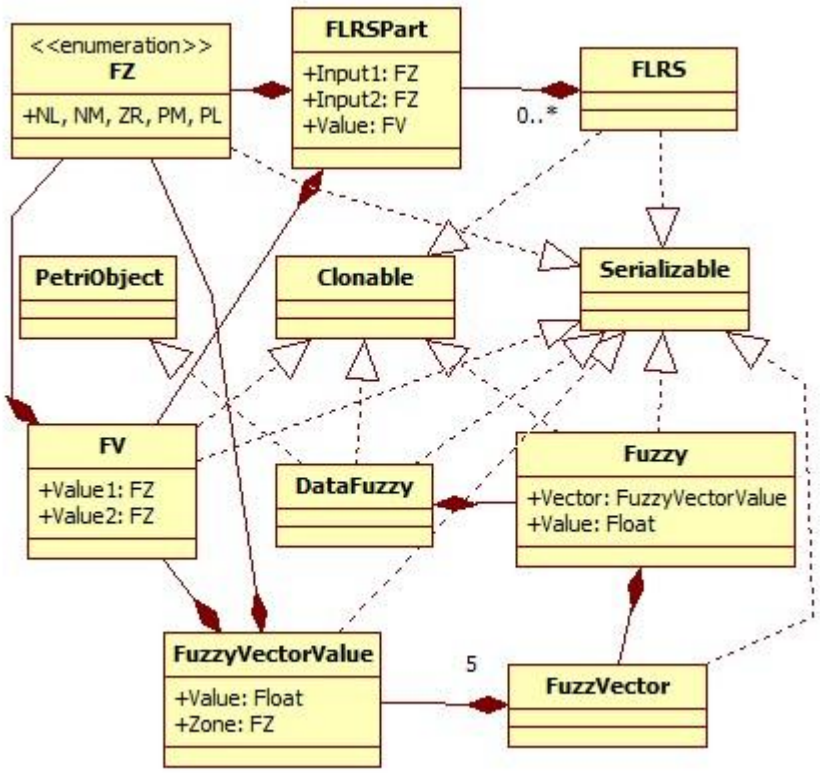


Figure 5.2 The FLETPN Token and TokenType

Implementation.

The code is implemented with eclipse and can be found here: https://github.com/dahliajanabi/All_Petri_Nets_Framework in the *DCS_FuzzyLab_1_2*, the *Inversor* class.

Code Explanation:

In code sequence 1, a *PetriNet* object is created and given a name (Inversor) so it can be shown on the *GUI* of the controller. A port number is assigned to each FLETPN so it can be connected to the network and receive tokens through its input channels.

```
PetriNet pn = new PetriNet();
pn.PetriNetName = "Inversor";

pn.NetworkPort = 1081;
```

The FLRS is created in the same order that is shown in the table 2. The rows are x1 values (p1) starting from NL- PL, the columns are for x2 values. The cell [0,0] is [x1 NL, x2 NL], that has the value for x3 (PL) and the value for x4 (PL). the first cell is highlighted below.

```
FLRS flrs2x2 = new FLRS(new FV(FZ.PL, FZ.PL), new FV(FZ.PL, FZ.PM), new FV(FZ.PL, FZ.ZR), new FV(FZ.PL, FZ.NM),new FV(FZ.PL, FZ.NL),
                        new FV(FZ.PM, FZ.PL), new FV(FZ.PM, FZ.PM), new FV(FZ.PM, FZ.ZR), new FV(FZ.PM, FZ.NM),new FV(FZ.PM, FZ.NL),
                        new FV(FZ.ZR, FZ.PL), new FV(FZ.ZR, FZ.PM), new FV(FZ.ZR, FZ.ZR), new FV(FZ.ZR, FZ.NM),new FV(FZ.ZR, FZ.NL),
                        new FV(FZ.NM, FZ.PL), new FV(FZ.NM, FZ.PM), new FV(FZ.NM, FZ.ZR), new FV(FZ.NM, FZ.NM),new FV(FZ.NM, FZ.NL),
                        new FV(FZ.NL, FZ.PL), new FV(FZ.NL, FZ.PM), new FV(FZ.NL, FZ.ZR), new FV(FZ.NL, FZ.NM),new FV(FZ.NL, FZ.NL));
```

The places are created by creating an object of DataFuzzy for each place, The initial value can be set using the *setValue*(*value*) method, then each place must be added to the *PlaceList* of the *PetriNet* object pn.

```
DataFuzzy p1 = new DataFuzzy();
p1.SetName("P1");
p1.SetValue(new Fuzzy(0.1F));
pn.PlaceList.add(p1);
```

The transitions are defined as *PetriTransition*, the *TransitionName* is the transition ID. The input places for the transition must be added.

```
PetriTransition t1 = new PetriTransition(pn);
             t1.TransitionName = "T1";
             t1.InputPlaceName.add("P1");
             t1.InputPlaceName.add("P2");
```

To define a guard for the transition, Condition objects must be created containing the transition ID, the input place, and the condition operation. The *Condition* objects must be linked with *LogicConnector* (AND, OR).

The implementation for guard t1: $p1 \neq \phi$ AND $p2 \neq \phi$

the first condition is connected to the second one with an AND, just like a *LinkList*.

```
Condition T1Ct1 = new Condition(t1, "P1", TransitionCondition.NotNull);
Condition T1Ct2 = new Condition(t1, "P2", TransitionCondition.NotNull);
T1Ct1.SetNextCondition(LogicConnector.AND, T1Ct2);
```

Also, a *GuardMapping* object must be created to add the first condition part, and to add the *Activations* (mappings).

```
GuardMapping grdT1 = new GuardMapping();
grdT1.condition = T1Ct1;
```

The Activation object must contain the transition ID, the FLRS object, the input Arraylist<PlaceNameWithWeight> that has for each item the place name, and the weight of the arc connecting the place to the transition, the transition operation (*FLRS*), the output ArrayList <String> that has the output places names. The *Activations* objects must be added to the *GuardMapping* list as well. And when it is finished, the entire list should be added to the transition t_1 to store the first guard.

```
ArrayList<PlaceNameWithWeight> input = new ArrayList<>();
                    input.add(new PlaceNameWithWeight("P1", 1F));
                    input.add(new PlaceNameWithWeight("P2", 2F));

                    ArrayList<String> twoOutput = new ArrayList<>();
                    twoOutput.add("P3");
                    twoOutput.add("P4");

                    grdT1.Activations.add(new Activation(t1, flrs2x2, input,
TransitionOperation.FLRS, twoOutput));

                    t1.GuardMappingList.add(grdT1);
```

The transition delay must be defined. Then the transition is added to the Petri net *Transitions* list.

```
                    t1.Delay = 0;
                    pn.Transitions.add(t1);
```

To run any Petri net and view the *GUI* for it as shown in fig. 5.3.

```
        pn.Delay = 3000;
        PetriNetWindow frame = new PetriNetWindow(false);
        frame.petriNet = pn;
        frame.setVisible(true);
```

The highlighted line in figure 5.3 shows the fuzzy token computed according to the weight that is on the arc that connects the place to the triggered transition. The rest of the fuzzy tokes are displayed by having a default weight 1.

Figure 5.4 shows the inputFuzzy, that can be executed as well from the GUIs package so you can send tokens during the execution time to the input channels of the FLETPN, you need to write the place name of the input channel (it is case sensitive), the float value, and the port number of the FLETPN.

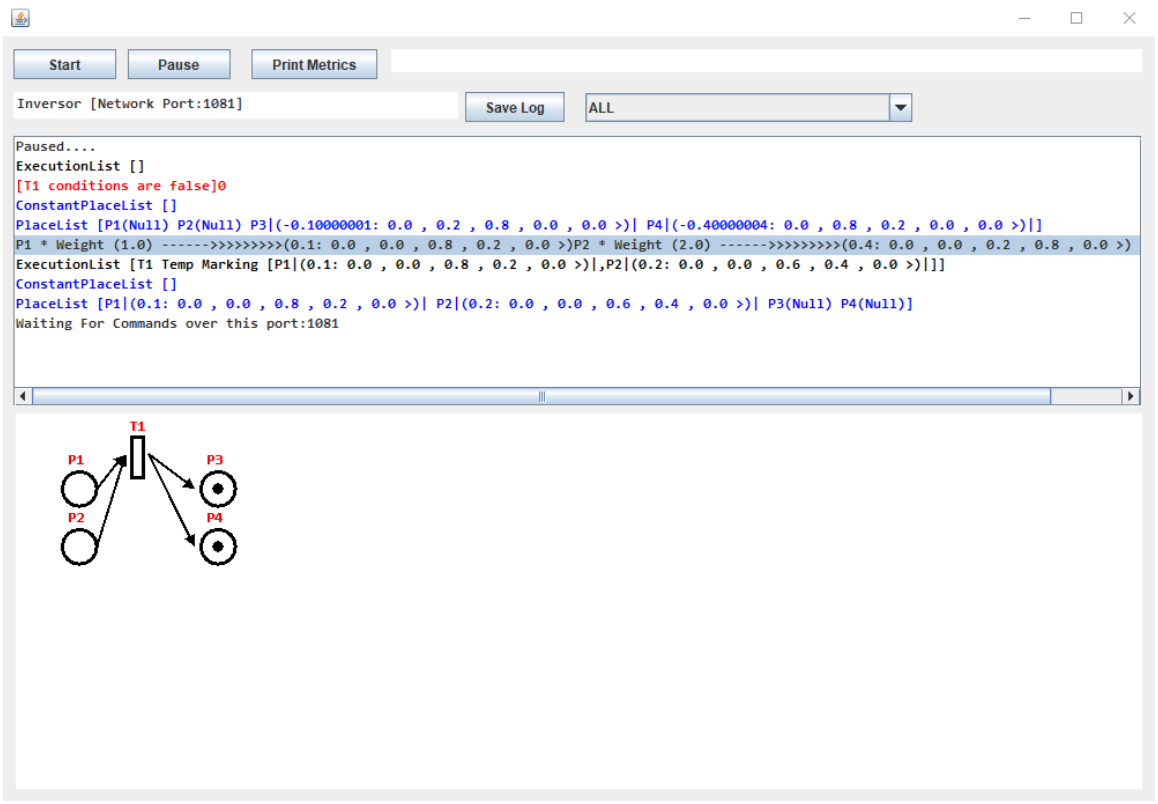


Figure 5.3 The GUI of the inversor

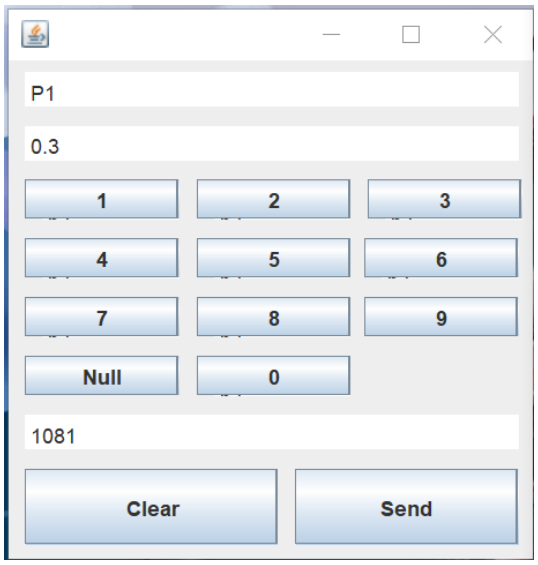


Figure 5.4 The GUI of the inputFuzzy

Code listing 1: Inversor

```
package DCS_FuzzyLab1_2;

import java.util.ArrayList;
import Components.Activation;
import Components.Condition;
import Components.GuardMapping;
import Components.PetriNet;
import Components.PetriNetWindow;
import Components.PetriTransition;
import DataObjects.DataFuzzy;
import DataOnly.FLRS;
import DataOnly.FV;
import DataOnly.Fuzzy;
import DataOnly.FuzzyVector;
import DataOnly.PlaceNameWithWeight;
import Enumerations.FZ;
import Enumerations.LogicConnector;
import Enumerations.TransitionCondition;
import Enumerations.TransitionOperation;

public class Inversor {
        public static void main (String[]args) {
        PetriNet pn = new PetriNet();
        pn.PetriNetName = "Main Petri";
        pn.NetworkPort = 1081;

        FLRS flrs2x2 = new FLRS(new FV(FZ.PL, FZ.PL), new FV(FZ.PL, FZ.PM), new FV(FZ.PL,
FZ.ZR), new FV(FZ.PL, FZ.NM),new FV(FZ.PL, FZ.NL),
                                                        new FV(FZ.PM, FZ.PL), new FV(FZ.PM,
FZ.PM), new FV(FZ.PM, FZ.ZR), new FV(FZ.PM, FZ.NM),new FV(FZ.PM, FZ.NL),
                                                        new FV(FZ.ZR, FZ.PL), new FV(FZ.ZR,
FZ.PM), new FV(FZ.ZR, FZ.ZR), new FV(FZ.ZR, FZ.NM),new FV(FZ.ZR, FZ.NL),
                                                        new FV(FZ.NM, FZ.PL), new FV(FZ.NM,
FZ.PM), new FV(FZ.NM, FZ.ZR), new FV(FZ.NM, FZ.NM),new FV(FZ.NM, FZ.NL),
                                                        new FV(FZ.NL, FZ.PL), new FV(FZ.NL,
FZ.PM), new FV(FZ.NL, FZ.ZR), new FV(FZ.NL, FZ.NM),new FV(FZ.NL, FZ.NL));

        flrs2x2.Print();

        DataFuzzy p1 = new DataFuzzy();
        p1.SetName("P1");
        p1.SetValue(new Fuzzy(0.1F));
        pn.PlaceList.add(p1);

        DataFuzzy p2 = new DataFuzzy();
        p2.SetName("P2");
        p2.SetValue(new Fuzzy(0.2F));
        pn.PlaceList.add(p2);

        DataFuzzy p3 = new DataFuzzy();
        p3.SetName("P3");
        pn.PlaceList.add(p3);

        DataFuzzy p4 = new DataFuzzy();
        p4.SetName("P4");
        pn.PlaceList.add(p4);

        // T1 ------------------------------------------------
                        PetriTransition t1 = new PetriTransition(pn);
```

```
                        t1.TransitionName = "T1";
                        t1.InputPlaceName.add("P1");
                        t1.InputPlaceName.add("P2");

                        Condition T1Ct1 = new Condition(t1, "P1",
TransitionCondition.NotNull);
                        Condition T1Ct2 = new Condition(t1, "P2",
TransitionCondition.NotNull);
                        T1Ct1.SetNextCondition(LogicConnector.AND, T1Ct2);

                        GuardMapping grdT1 = new GuardMapping();
                        grdT1.condition = T1Ct1;

                        ArrayList<PlaceNameWithWeight> input = new ArrayList<>();
                        input.add(new PlaceNameWithWeight("P1", 1F));
                        input.add(new PlaceNameWithWeight("P2", 2F));

                        ArrayList<String> twoOutput = new ArrayList<>();
                        twoOutput.add("P3");
                        twoOutput.add("P4");

                        grdT1.Activations.add(new Activation(t1, flrs2x2, input,
TransitionOperation.FLRS, twoOutput));

                        t1.GuardMappingList.add(grdT1);

                        t1.Delay = 0;
                        pn.Transitions.add(t1);

                        // ------------------------------------------

                        System.out.println("Exp1 started \n ------------------------------");
                        pn.Delay = 3000;
                        // pn.Start();

                        PetriNetWindow frame = new PetriNetWindow(false);
                        frame.petriNet = pn;
                        frame.setVisible(true);
        }
}
```

Exercises:

1. Test the application for different values of x_1,x_2, w_1 and w_2 and display x_3 and x_4.
2. Starting from the previous application, it is required to build an application that will work as an adder/ subtractor and will calculate the output values x_3 and x_4.

The mathematical model of the adder / subtractor component:

$(x_1, x_2) \rightarrow (x_3, x_4)$ $x_3 = x_1 + x_2$; $x_4 = x_1 - x_2$; $w_1 = w_2 = 1$:

$x_3 = w_1 \cdot x_1 + w_2 \cdot x_2$; $\leftrightarrow (w_1 \cdot x_1 + w_2 \cdot x_2) \in [-1,1]$

$x_3 = -1;$ $\leftrightarrow (w_1 \cdot x_1 + w_2 \cdot x_2) < -1$

$x_3 = 1;$ $\leftrightarrow (w_1 \cdot x_1 + w_2 \cdot x_2) > 1$

$x_4 = w_1 \cdot x_1 - w_2 \cdot x_2$; $\leftrightarrow (w_1 \cdot x_1 - w_2 \cdot x_2) \in [-1,1]$

$x_4 = -1;$ $\leftrightarrow (w_1 \cdot x_1 - w_2 \cdot x_2) < -1$

$x_4 = 1;$ $\leftrightarrow (w_1 \cdot x_1 - w_2 \cdot x_2) > 1$

and the FLRS table:

Table 3: Adder/Subtractor FLRS Table

$x_1 \setminus x_2$	NL	NM	ZR	PM	PL
NL	NL, ZR	NL, NM	NL, NL	NM, NL	ZR, NL
NM	NL, PM	NL, ZR	NM, NM	ZR, NL	PM, NL
ZR	NL, PL	NM, PM	ZR, ZR	PM, NM	PL, NL
PM	NM, PL	ZR, PL	PM, PM	PL, ZR	PL, NM
PL	ZR, PL	PM, PL	PL, PL	PL, PM	PL, ZR

The class diagram is required.

Test for different values of x_1, x_2, w_1 and w_2.

3. Compute by hand using the FLRS from table3 (adder/subtractor): $x_1 = 0.2$, $w_1 = 1$, $x_2 = 0.3$, $w_2 = 2$. For verification: $x_3 = 0.8$, $x_4 = -0.4$.

6. Verification of knowledge

1) What are fuzzy sets and how do they work with them?

2) How to change from real variables to fuzzy variables?

3) Explain what is a Fuzzy Logic Enhanced Time Petri Nets and what are they used for?

4) What is a token set in a place p_i on the FLETPN network?

5) What is the usefulness of the Fuzzy Logic Rule Set tables?